

Universidad ORT Uruguay

Facultad De Ingeniería

Informe Final

Integrantes:

Nicolas Bidenti (305108)
Santiago Canadell (282542)
Felipe Delgado (281987)

Tutores:

Santiago Pablo Tonarelli Alvarez
Gastón Antonio Mousques Anaya
Pablo Marcelo Geymonat Roldan

Repositorio:

https://github.com/IngSoft-AR/305108_282542_281987

ÍNDICE

Introducción y alcance	3
Antecedentes	3
Propósito del sistema	3
Requerimientos significativos de arquitectura	3
Resumen de requerimientos de atributos de calidad	4
Evidencia de performance medida	6
Definición de carga para el escalado (R8)	7
Documentación de la arquitectura	7
Diagrama de contexto	7
Estilo arquitectónico y decisión global	8
Vista de Módulos	8
Vista de descomposición	9
Vista de uso	9
Vista de capas	9
Catálogo de elementos	9
Decisiones de diseño	10
Comportamiento	10
Una nota sobre la notación	10
Catálogo de elementos	10
Interfaces	12
Relación con elementos lógicos	12
Comportamiento: tres flujos clave	12
Pipeline GPS	12
Clasificación de reserva con IA	13
Pago con circuit breaker	13
Vista de Despliegue	14
Catálogo de elementos	14
Persistencia y conexión externa	15
Vista de Instalación	15
Decisiones de arquitectura (ADRs)	119
Sobre el proveedor de pagos	19
Cómo el sistema resiste las fallas	19
Deuda técnica y mejoras identificadas	20
Anexo	21
Catálogo completo de ADRs	21
Comparativa de las tres estrategias de clasificación (R10)	22
Pruebas de carga con K6	22
Carga de datos: los scripts de seed	23
Gestión del proyecto y control de versiones	23
Uso de agentes de inteligencia artificial	24

Introducción y alcance

MOVE es una empresa de traslados que necesita mover bienes dentro de la ciudad, desde medicamentos y documentación confidencial hasta equipos electrónicos o equipaje, y quería un sistema que le permitiera gestionar todo el ciclo: que un cliente reserve un traslado, que se cotice y se pague, que un operador asigne un vehículo y un conductor, y que el viaje se monitoree en tiempo real con la posición que envían los GPS instalados en los vehículos. Sobre esa última parte está puesto el foco del negocio, porque es donde aparecen las situaciones que requieren intervención: un vehículo que entra a una zona peligrosa, o que queda detenido más tiempo del razonable.

El sistema distingue dos tipos de cliente. Las empresas trasladan siempre productos parecidos, así que pre registran sus bienes y los asocian a las categorías que define MOVE, lo que hace que sus reservas se procesen rápido. Los clientes particulares, en cambio, describen el bien con texto libre, y el sistema tiene que deducir la categoría a partir de esa descripción. Esa categoría no es un dato cualquiera: determina el comportamiento del traslado, porque según el tipo de bien puede hacer falta monitoreo especial, generar alertas ante desvíos o aplicar recargos en el costo.

Construimos el sistema con un enfoque backend-first, que es lo que pide la letra. Toda la funcionalidad vive en servicios de backend que se prueban con una colección de Postman, y los flujos no dependen de ninguna interfaz gráfica para funcionar. Sumamos igualmente una pequeña aplicación web de mapas (frontend-geo) para que la demo del monitoreo geográfico se vea mejor, pero es complementaria: el sistema funciona completo sin ella. Tampoco construimos dispositivos GPS reales, porque la letra no lo exige; en su lugar hicimos un simulador que envía puntos de ubicación al mismo endpoint que usaría un dispositivo real, de modo que para el sistema no hay diferencia entre un GPS físico y el simulado.

Este documento describe la arquitectura que diseñamos siguiendo el modelo Views & Beyond. Después de presentar los requerimientos que más influyeron en el diseño y el estilo arquitectónico que elegimos, mostramos tres vistas de la arquitectura (módulos, componentes y conectores, y despliegue), cada una con su diagrama, su catálogo de elementos y las decisiones que la sustentan. Cerramos con las decisiones de arquitectura registradas como ADRs y con las tácticas de resiliencia que aplicamos. El objetivo es que otro arquitecto pueda entender, a partir de este documento, no solo qué construimos sino por qué tomamos cada decisión.

Antecedentes

Propósito del sistema

Los usuarios del sistema son cinco roles: el cliente empresa y el cliente particular, que crean reservas y pagan; el conductor, que ejecuta el traslado y reporta incidencias; el operador, que monitorea los traslados, gestiona alertas y asigna recursos; y el administrador, que gestiona la configuración del sistema. El detalle completo de las funcionalidades está en la letra del obligatorio; acá resumimos los elementos clave que tienen impacto en la arquitectura.

Requerimientos significativos de arquitectura

Resumen de requerimientos funcionales:

ID	Descripción	Actor
F1	Registro de cliente (empresa o particular)	Cliente
F2	Ingreso mediante autenticación externa	Cliente
F3	Preregistro de productos y orígenes frecuentes	Cliente Empresa
F4.1	Crear reserva con clasificación automática del bien por descripción	Cliente Particular
F4.2	Crear reserva usando datos preregistrados	Cliente Empresa
F5	Cotizar el costo del traslado según distancia y categoría	Cliente
F6	Confirmar y pagar mediante pasarela externa	Cliente
F7	Consultar reservas con filtros por fecha y estado	Cliente
F8	Configurar categorías y reglas de comportamiento	Administrador
F9	Gestión de zonas geográficas (rojas y preferentes)	Administrador
F10	Gestión de vehículos	Administrador
F11	Gestión de usuarios	Administrador
F12	Asignar vehículo y conductor a una reserva	Operador
F13	Iniciar traslado	Conductor
F14	Recibir y verificar puntos GPS	Sistema
F15	Detectar situaciones relevantes (zona roja, detención)	Sistema
F16	Generar alertas asociadas al traslado	Sistema
F17	Finalizar traslado	Conductor
F18	Consultar traslados en curso con filtros	Operador
F19	Clasificación manual de reservas no reconocidas	Operador

Resumen de requerimientos de atributos de calidad

La forma de la arquitectura de MOVE no salió de una preferencia estética sino de un conjunto de requerimientos no funcionales que tironeaban en direcciones distintas, y que tuvimos que balancear. Antes de mostrar las vistas, conviene entender cuáles fueron esos

requerimientos, porque cada decisión que tomamos más adelante responde a uno o varios de ellos.

El primero, y el que más condicionó todo, fue la **disponibilidad de los flujos críticos (R7)**. La letra es explícita: el grupo de funcionalidades que va del ingreso al pago (F2 a F6) y el grupo del traslado en curso (F13 a F15) no pueden verse afectados por fallas en otras partes del sistema. Eso nos empujó desde el principio a separar el sistema en servicios independientes, de modo que si uno se cae no arrastre a los demás. Un cliente tiene que poder pagar su reserva aunque el monitoreo GPS esté caído, y un conductor tiene que poder transmitir su posición aunque el servicio de reservas no responda.

El segundo grupo de requerimientos fue de **performance (R1, R2, R3)**, y acá la letra pone números concretos. Las reservas de los veinte mejores clientes empresa tienen que quedar listas para pagar en menos de 600 milisegundos, las consultas tienen que responder en menos de 300 y los listados en menos de 500, y cada punto GPS tiene que procesarse de punta a punta en menos de 5 segundos. Estos números nos obligaron a pensar dónde poner caches, qué procesar de forma asíncrona para no bloquear, y qué índices necesitábamos en la base de datos.

El tercero fue la **clasificación de bienes con IA (R10)**, que es un requerimiento particular porque no pide solo que funcione, sino que evaluemos y comparemos tres estrategias distintas (búsqueda semántica, IA generativa local, y una tercera a criterio nuestro) midiendo tiempo de respuesta, precisión y simplicidad. Esto nos llevó a diseñar la clasificación como una cascada de estrategias intercambiables.

El resto de los requerimientos completó el cuadro: la **escalabilidad de hasta 50 veces la carga (R8)**, la **observabilidad con métricas y dashboards (R4)**, la **seguridad con registro de accesos (R6)**, la **portabilidad con contenedores (R9)**, la posibilidad de **simular y probar el sistema, incluido el GPS (R5)**, y la **interoperabilidad con servicios externos** de autenticación, pagos y geolocalización, sabiendo que el proveedor de pagos puede cambiar.

La siguiente tabla resume cada requerimiento no funcional, el atributo de calidad que impacta, y la respuesta de diseño con la que lo abordamos. Las tácticas concretas se desarrollan en las vistas y en los ADRs.

ID Requerimiento	ID Requerimiento de Calidad o restricción	Descripción
F4.2	R1 Performance	La reserva de un cliente empresa del top-20 debe quedar lista para pagar en menos de 600ms; las de no frecuentes entre 700 y 1000ms
F7, F18	R2 Performance	Las consultas deben responder en menos de 300ms y los listados en menos de 500ms bajo carga máxima
F14, F15, F16	R3 Performance	Cada punto GPS debe procesarse de punta a punta, incluida la detección de situaciones, en menos de 5s
Todas	R4 Observabilidad	El sistema debe permitir verificar su estado y visualizar métricas

F14, todas	R5 Testabilidad	El sistema debe poder validarse con pruebas y simulaciones, incluida la del GPS
F2, todas	R6 Seguridad	Se deben registrar los accesos autorizados y no autorizados
F2-F6, F13-F15	R7 Disponibilidad	Los flujos críticos deben seguir operando ante fallas parciales en otras partes del sistema
Todas	R8 Escalabilidad	El sistema debe soportar incrementos de carga de hasta 50 veces
Todas	R9 Portabilidad	El sistema debe poder desplegarse en cualquier plataforma mediante contenedores
F4.1	R10 Modificabilidad	La clasificación del bien debe permitir evaluar y comparar tres estrategias por tiempo, precisión y simplicidad

Más adelante, en cada vista y en los ADRs, desarrollamos la táctica concreta con la que respondimos a cada uno de estos requerimientos.

Evidencia de performance medida

Para no quedarnos en la promesa de que el sistema cumple los tiempos, medimos los escenarios con K6 sobre el stack Docker corriendo localmente, cinco minutos por escenario. Los resultados quedaron bastante por debajo de los umbrales que pide la letra, con cero errores. Aclaramos que estos números dependen del hardware donde corre la prueba, así que los tomamos como evidencia de que la arquitectura no introduce cuellos de botella, no como una garantía absoluta de producción.

Escenario	Umbral (R)	p95 medido
Reserva empresa top-20 (R1)	< 600ms	69.9ms
Reserva empresa no frecuente (R1)	< 1000ms	71.61ms
Consultas (R2)	< 300ms	19.43ms
Listado de traslados (R2)	< 500ms	14.75ms
Pipeline GPS (R3)	< 5000ms	19.94ms
50x (R8)	reservas <600 / GPS <5000	12.41ms / 17.44ms

Para verificar el escalado de R8 corrimos además un escenario de alta demanda con cincuenta veces la carga base (quinientas reservas por minuto más doscientos cincuenta dispositivos GPS enviando puntos en simultáneo), y el resultado fue que con una sola instancia de cada servicio el sistema mantuvo un p95 de 12.41 milisegundos en reservas y 17.44 milisegundos en GPS, con cero errores sobre más de doscientos mil requests. Es decir, en este hardware la carga de prueba ni siquiera llega a saturar una instancia. Eso confirma que el escalado horizontal que describe R8 es una capacidad disponible de la arquitectura (los servicios no guardan estado en memoria) más que una necesidad para cumplir los tiempos bajo esta carga.

El caso del listado de traslados (F18) es el que más mejoró. En el diseño original ese listado resolvía la consulta con un JOIN de cinco tablas repartidas en tres esquemas distintos, lo que era lento y además acoplaba un servicio con los datos de otro. Lo cambiamos para que cada servicio mantenga una proyección local de los datos que necesita de los demás, alimentada por eventos. El listado pasó a leer de una sola tabla local indexada, y el p95 quedó en 8.7 milisegundos, muy debajo del medio segundo que pide la letra. Esa decisión la registramos en el ADR-023 y la retomamos al hablar de resiliencia.

Definición de carga para el escalado (R8)

R8 nos pide soportar incrementos de hasta cincuenta veces la carga en períodos de alta demanda, y nos exige definir explícitamente qué entendemos por baja y por alta demanda. Tomamos como baja demanda el escenario de operación normal que la propia letra describe en R2: cien reservas por minuto con cincuenta conductores transmitiendo su posición en simultáneo, lo que da unos trescientos puntos GPS por minuto. Ese es nuestro baseline, el punto donde el sistema tiene que cumplir todos los tiempos de respuesta de R1, R2 y R3 con una sola instancia de cada servicio.

La alta demanda la definimos como cincuenta veces ese baseline, es decir, alrededor de cinco mil reservas por minuto con dos mil quinientos conductores transmitiendo. Ahora bien, el número exacto donde el sistema empieza a degradarse depende del hardware en el que corra, así que más que fijar un valor de laboratorio, lo tratamos como un objetivo a verificar empíricamente con K6: aumentamos la carga gradualmente hasta que algún tiempo de respuesta se viola, escalamos horizontalmente el servicio afectado con un comando de Docker Compose, y comprobamos que los tiempos vuelven al baseline. La idea de fondo de R8 no es que una sola instancia aguante cincuenta veces la carga, sino que la arquitectura permita absorber ese crecimiento agregando instancias, y eso es posible porque los servicios no guardan estado en memoria (el estado vive en Redis y Postgres), el tracking usa hashing consistente por dispositivo para que escalar no desordene los puntos de un mismo GPS.

Documentación de la arquitectura

En las secciones que siguen documentamos la arquitectura de MOVE desde tres puntos de vista, como propone el modelo Views & Beyond: la vista de módulos, que muestra cómo está organizado el código; la vista de componentes y conectores, que muestra el sistema en ejecución; y la vista de despliegue, que muestra cómo se asigna a la infraestructura. Antes de entrar en cada una, ubicamos al lector con un diagrama de contexto que delimita el sistema.

Diagrama de contexto

El diagrama de contexto muestra a MOVE como una sola caja y a su alrededor todos los actores y sistemas externos con los que interactúa, indicando el tipo de conexión de cada uno. Sirve para entender de un vistazo dónde están los límites del sistema: qué es parte de MOVE y qué queda afuera.

[Click aquí para ver imagen](#)

Del lado de los actores humanos, los clientes (empresa y particular), el conductor, el operador y el administrador acceden al sistema por HTTPS, siempre a través del API gateway, que es el único punto de entrada. Los dispositivos GPS instalados en los vehículos también entran por la misma vía: envían sus puntos de ubicación al sistema por HTTPS. En nuestro caso esos dispositivos están simulados, pero desde el punto de vista del sistema son un emisor externo igual que un GPS real, porque usan el mismo endpoint.

Del otro lado están los tres servicios externos que MOVE consume. Firebase valida los tokens de autenticación en cada request. La pasarela de pago procesa los cobros, y la

conectamos a través de un adapter intercambiable que en esta entrega es un mock. OpenRouteService calcula las distancias entre origen y destino para la cotización. La única comunicación que sale del sistema hacia un actor humano es la notificación en tiempo real al operador mediante Server-Sent Events, que es como le llegan las alertas de los traslados en curso.

Estilo arquitectónico y decisión global

Cuando arrancamos a diseñar, la primera decisión grande fue cómo descomponer el sistema, y la tomamos mirando el requerimiento de disponibilidad (R7). La letra nos pedía que el flujo de reservas y pagos, por un lado, y el flujo del traslado en curso, por el otro, pudieran seguir funcionando aunque una parte del sistema fallara. Un monolito no nos servía para eso, porque una falla en el procesamiento de GPS, que es la parte más exigida, podía tirar también el flujo de pagos, y eso es justo lo que R7 prohíbe. Así que descartamos el monolito por una razón concreta de disponibilidad.

En el otro extremo estaban los microservicios puros, con una base de datos por servicio y transacciones distribuidas. También los descartamos, pero por una razón distinta: para un equipo de tres personas y el alcance de este obligatorio, la complejidad de coordinar transacciones entre servicios era desproporcionada respecto al beneficio. Habríamos pasado más tiempo peleando con la consistencia distribuida que construyendo funcionalidad.

Elegimos un punto intermedio que llamamos arquitectura orientada a servicios híbrida. Separamos el sistema en tres servicios HTTP según los grupos de funcionalidad que la letra ya venía agrupando: uno para reservas y clientes, otro para operaciones (zonas, vehículos, asignaciones), y otro para el traslado y el GPS. Sumamos dos procesos worker que corren aparte para el trabajo pesado y asíncrono, y un API gateway como único punto de entrada. Cada servicio vive en su propio contenedor, así que un crash en uno no arrastra a los demás, que es exactamente lo que R7 necesitaba. Pero compartimos una única instancia de base de datos con un esquema separado por servicio, lo que nos evitó la complejidad de las transacciones distribuidas sin perder el aislamiento lógico de los datos. Esta decisión global la registramos en el ADR-001.

Sobre esa base combinamos tres estilos, cada uno donde resolvía mejor un problema concreto. La comunicación entre servicios es **dirigida por eventos** a través de RabbitMQ, porque los servicios no pueden llamarse por HTTP entre sí sin acoplarse y romper R7. El procesamiento de los puntos GPS sigue un estilo **Pipes and Filters**, una secuencia de filtros independientes encadenados por colas, porque el procesamiento del GPS tiene etapas bien diferenciadas (validar, enriquecer, persistir, detectar situaciones, alertar) que conviene poder modificar por separado. Y dentro de cada servicio aplicamos **Clean Architecture** en cuatro capas, para que la lógica de negocio no dependa de la base de datos ni del framework web, lo que nos permitió testear el dominio sin levantar infraestructura y cambiar adapters externos sin tocar el núcleo.

El resultado es un sistema de trece contenedores que se levanta con un solo comando. En las próximas tres secciones lo mostramos desde tres ángulos: cómo está organizado el código (vista de módulos), qué componentes lo forman en ejecución y cómo se comunican (vista de componentes y conectores), y cómo se despliega sobre la infraestructura (vista de despliegue).

Vista de Módulos

Esta vista muestra cómo organizamos el código fuente: cómo está dividido en paquetes y cómo se relacionan entre sí en tiempo de compilación. Responde a la pregunta de un desarrollador que se suma al proyecto y quiere saber dónde vive cada cosa y qué puede importar a qué.

[Click aqui para ver imagen](#)

Vista de descomposición

El sistema se descompone en seis paquetes. El paquete `@move/shared` concentra los tipos y contratos que comparten los servicios. Los tres servicios HTTP (`booking-service`, `operations-service` y `tracking-service`) agrupan cada uno un conjunto de funcionalidades de la letra. El `ai-worker` corre la clasificación con IA de forma asíncrona, y el `gps-simulator` es la herramienta que usamos para generar tráfico de prueba. Esta descomposición no es arbitraria: cada servicio agrupa las funcionalidades que la letra ya venía agrupando por rol y por flujo, lo que nos permite que cada uno tenga su propio ciclo de vida y se pueda desplegar y escalar por separado.

[Click aqui para ver idemagen](#)

Vista de uso

Dentro de cada servicio, las cuatro capas se usan en una sola dirección: `presentation` usa `application`, `application` usa `domain`, e `infrastructure` usa tanto `application` como `domain`. El `domain` no usa ninguna otra capa, es el núcleo. Hacia afuera del servicio, la única dependencia es que la capa de `infrastructure` usa el paquete `@move/shared`, que es donde viven los contratos de los eventos de RabbitMQ y las constantes de las colas. Este patrón de uso es lo que mantiene el acoplamiento bajo control: la lógica de negocio del `domain` no conoce ni a Prisma, ni a Express, ni a ningún detalle de infraestructura, así que pudimos testearla sin levantar base de datos ni servidor.

[Click aqui para ver imagen](#)

Vista de capas

Las mismas cuatro capas, vistas como agrupamiento por responsabilidad. La capa de `presentation` se ocupa de los controllers, las rutas y los DTOs. La de `application` contiene los casos de uso, uno por cada intención del usuario. La de `domain` tiene las entidades, los `value objects` y los puertos (las interfaces que el dominio define y que la infraestructura implementa). Y la de `infrastructure` tiene los adapters concretos: los repositorios de Prisma, los clientes de Redis y RabbitMQ, el adapter de Firebase, y los demás. El `domain` queda como la capa más interna, hacia la que apuntan todas las dependencias. Es justamente esta separación la que hace posible cambiar un adapter externo, como el de pagos, sin tocar el núcleo, que es lo que describimos en el patrón Ports y Adapters (ADR-022).

[Click aqui para ver imagen](#)

Catálogo de elementos

El código vive en un monorepo gestionado con `pnpm workspace`, que agrupa seis paquetes. Elegir un monorepo nos permitió compartir tipos y contratos entre servicios sin publicar paquetes a un registro externo, y mantener todo el historial en un solo lugar, que para un equipo de tres es más práctico que coordinar varios repositorios.

Paquete	Responsabilidad	Funcionalidades
<code>@move/shared</code>	Tipos compartidos, contratos de los eventos de RabbitMQ, constantes de las colas y middleware de auditoría y trazas	Transversal
<code>booking-service</code>	Registro y login de clientes, preregistro de bienes de empresa, creación de reservas, cotización, pago y consulta, y notificación al operador	F1 a F7, F19

operations-service	Categorías y reglas, zonas geográficas, vehículos, usuarios, asignación de vehículo y conductor, y consulta de traslados en curso	F8 a F12, F18
tracking-service	Inicio y fin del traslado, recepción y procesamiento de los puntos GPS, detección de situaciones, alertas e incidencias	F13 a F17
ai-worker	Clasificación de bienes con el modelo de lenguaje, de forma asíncrona	R10
gps-simulator	Herramienta de línea de comandos que simula los puntos de un GPS para pruebas y demo	R5

Decisiones de diseño

La regla que gobierna las cuatro capas es el corazón de Clean Architecture: las dependencias apuntan siempre hacia adentro, hacia el dominio. Tomamos esta decisión porque nos dio dos cosas que necesitábamos. Primero, pudimos testear la lógica de negocio sin levantar base de datos ni servidor, porque el dominio no sabe nada de Prisma ni de Express. Segundo, pudimos cambiar implementaciones de infraestructura, por ejemplo el proveedor de pagos o el de geolocalización, sin tocar el núcleo, porque las capas externas implementan interfaces que define el dominio.

Las decisiones de arquitectura que sustentan esta vista son el ADR-009 (una base de datos compartida con un esquema por servicio, en lugar de una base por servicio) y el ADR-022 (Ports y Adapters para los servicios externos).

Comportamiento:

El comportamiento de los módulos se documenta en la sección de Componentes y Conectores mediante tres diagramas de secuencia: el pipeline GPS, la clasificación de reserva con IA y el pago con circuit breaker. Cada paquete del catálogo anterior participa en al menos uno de esos flujos: booking-service en la clasificación y el pago, tracking-service en el pipeline GPS, ai-worker en la clasificación asíncrona, y @move/shared como contrato compartido en los tres.

Vista de Componentes y Conectores

Esta vista muestra el sistema en ejecución: qué componentes lo forman cuando está corriendo y por qué medios se comunican entre sí. Es la vista donde se ven las decisiones de resiliencia y de comunicación que tomamos pensando en R7 y R8.

Una nota sobre la notación

Representamos cada componente como un paquete en lugar de usar la notación de interfaces provistas y requeridas (lollipop y socket), por legibilidad y por compatibilidad con nuestra herramienta de modelado. Las interfaces que median las comunicaciones (los Ports de los servicios externos) no se pierden: las documentamos en la tabla de interfaces de esta misma vista y en el ADR-022.

[Click aquí para ver imagen](#)

Catálogo de elementos

La siguiente tabla describe cada componente y cada tipo de conector de la vista.

Componente / conector	Tipo	Descripción
-----------------------	------	-------------

nginx (api-gateway)	Componente	Punto de entrada único. Rutea cada petición al servicio según la ruta. Es un proxy inverso puro, sin lógica de negocio ni validación de tokens (ADR-016, ADR-019)
booking-service	Componente	Servicio HTTP de reservas y clientes (F1 a F7, F19)
operations-service	Componente	Servicio HTTP de operaciones: zonas, vehículos, asignaciones, traslados en curso (F8 a F12, F18)
tracking-service	Componente	Servicio HTTP de traslados y GPS, contiene el pipeline de siete etapas (F13 a F17)
ai-worker	Componente	Proceso aparte, sin puerto HTTP, que clasifica con el modelo de lenguaje de forma asíncrona
PostgreSQL	Componente	Base de datos con PostGIS (geometría) y pgvector (embeddings)
Redis	Componente	Cache, colas Bull y canal Pub/Sub
RabbitMQ	Componente	Broker de eventos entre servicios
Ollama	Componente	Motor de IA local: embeddings y modelo de lenguaje
Firebase, Pasarela de pago, OpenRouteService	Componente (adapter externo)	Servicios externos consumidos a través de un Port intercambiable
Prometheus, Grafana	Componente	Recolección de métricas y dashboards
HTTP REST	Conector	Síncrono, del gateway hacia los servicios. La cara pública del sistema
Eventos RabbitMQ	Conector	Asíncrono, entre servicios. Ningún servicio llama a otro por HTTP (ADR-003)
Colas Bull (Redis)	Conector	Asíncrono, dentro de un servicio. Encadena las etapas del pipeline GPS y los trabajos del ai-worker (ADR-002, ADR-003)
SSE	Conector	Del sistema hacia el operador, para notificaciones en tiempo real (ADR-017)

El de eventos asíncronos sobre RabbitMQ es el que comunica a los servicios entre sí, y es una de las decisiones más importantes de la arquitectura: los servicios nunca se llaman por HTTP directamente. Cuando booking necesita avisar que una reserva fue confirmada, no llama a operations, sino que publica un evento que operations consume cuando puede. Lo hicimos así por R7: si operations está caído, booking publica el evento igual y RabbitMQ lo retiene hasta que operations vuelva, en lugar de fallar la operación del cliente. El precio es consistencia eventual, que se ve en que cada servicio mantiene proyecciones locales de los datos de los demás.

Al ai-worker lo separamos del booking-service por una razón de tiempos. Clasificar un bien con el modelo de lenguaje puede tardar varios segundos, y eso no puede bloquear el request del cliente que está creando una reserva. Al ponerlo en un proceso independiente, el cliente recibe su respuesta de inmediato y la clasificación ocurre después, en background.

Interfaces

Los servicios externos se consumen a través de interfaces (Ports) que define el dominio, no a través de sus SDKs directamente. Esto es lo que permite cambiar de proveedor sin tocar la lógica de negocio.

Interfaz (Port)	Componente que la provee
IAuthService	Adapter de Firebase
IPaymentGateway	Adapter de la pasarela de pago (mock en esta entrega)
IGeolocationService	Adapter de OpenRouteService
ICategorizador	Las tres estrategias de clasificación (reglas, embeddings, modelo de lenguaje)
IServiceEmail	Adapter de correo (Nodemailer)

Relación con elementos lógicos

Cada componente de esta vista se construye a partir de los paquetes que documentamos en la vista de módulos. La tabla muestra esa correspondencia.

Componente	Paquetes que lo forman
booking-service	domain, application, infrastructure, presentation del paquete booking-service, más @move/shared
operations-service	domain, application, infrastructure, presentation del paquete operations-service, más @move/shared
tracking-service	domain, application, infrastructure, presentation del paquete tracking-service (el pipeline GPS vive en infrastructure), más @move/shared
ai-worker	el paquete ai-worker, más @move/shared
nginx (api-gateway)	configuración de nginx (no es código de la aplicación)

Comportamiento: tres flujos clave

Una vista de componentes y conectores cuenta qué piezas hay, pero no cómo colaboran a lo largo del tiempo. Para eso agregamos tres diagramas de secuencia que muestran los flujos donde se concentran las decisiones de diseño más interesantes. Cada uno representa un escenario concreto, y lo decimos explícitamente, porque un diagrama de secuencia siempre modela un caso y no todos a la vez.

Pipeline GPS

[Click aquí para ver imagen](#)

Este diagrama muestra el escenario de monitoreo normal: un punto GPS de un vehículo que va bien, fuera de zonas rojas y en movimiento, que por lo tanto no dispara ninguna alerta. Lo elegimos como caso base por ser el más frecuente y el más simple de seguir.

Lo que se ve es la decisión central del procesamiento GPS. Cuando llega un punto, el tracking-service no lo procesa en el momento: lo encola y responde de inmediato con un 202. Todo el trabajo pesado (validar que el dispositivo esté registrado y tenga un traslado activo, enriquecer el punto con el identificador del traslado, persistirlo, y analizarlo para detectar situaciones) ocurre después, fuera del request, en una secuencia de etapas sobre colas. Hicimos esto porque un dispositivo envía un punto cada diez segundos, y si procesáramos cada punto de forma síncrona dentro del request (con su consulta geográfica, su persistencia y sus dos análisis) acoplaríamos la latencia de recepción a la de la base de datos y de RabbitMQ. Al desacoplar, la recepción es siempre rápida y el procesamiento es resiliente.

Por legibilidad representamos las siete etapas del pipeline como un único elemento con sus pasos anotados, en lugar de dibujar siete participantes con todos sus encolados intermedios. El detalle de las etapas (validación, enriquecimiento, persistencia, geofence, detección de parada, generación de alerta y publicación) está descrito en el texto de esta vista y en el ADR-002. En el escenario con alerta, que no dibujamos para no recargar, el flujo es el mismo hasta la detección, y desde ahí la etapa de generación persiste la alerta en la base y la siguiente la publica en RabbitMQ para que el resto del sistema se entere.

Clasificación de reserva con IA

[Click aquí para ver imagen](#)

Este diagrama muestra el escenario donde la descripción del bien no se pudo clasificar con las estrategias rápidas y hay que recurrir al modelo de lenguaje. Es el caso que ejerce la cascada completa, por eso lo elegimos.

Cuando un cliente particular crea una reserva, el sistema intenta clasificar cada bien con una cascada de estrategias que corren de forma síncrona dentro del request, ordenadas de la más barata a la más cara. Primero una estrategia basada en reglas que busca palabras clave, y después una de búsqueda semántica con embeddings. Si alguna alcanza la confianza mínima, la reserva queda lista para cotizar y el flujo termina ahí, rápido. Pero si ninguna lo logra, la reserva queda en estado pendiente de clasificación, el sistema publica un evento, encola un trabajo para el ai-worker y notifica al operador. El modelo de lenguaje, que es la estrategia más lenta, corre entonces de forma asíncrona en el ai-worker, y cuando termina publica un evento que booking consume para asignar la categoría y avanzar la reserva.

Acá hay un detalle de honestidad que vale la pena decir. La cascada síncrona corre dos estrategias en este orden: primero la basada en reglas, después los embeddings. El modelo de lenguaje no forma parte de la cascada síncrona, sino que vive aparte en el ai-worker y solo entra cuando las dos primeras no alcanzaron. Lo decimos porque parte de nuestra propia documentación interna había quedado con el orden invertido, y al auditar el código confirmamos que el orden real es el que describimos aquí. La justificación del diseño en cascada está en el ADR-007, y la comparación de las tres estrategias por tiempo, precisión y simplicidad que pide R10 está en el anexo.

Pago con circuit breaker

[Click aquí para ver imagen](#)

Este diagrama muestra los tres caminos posibles al pagar una reserva, porque acá lo interesante es justamente cómo se comporta el sistema cuando las cosas salen mal. El circuit breaker envuelve la pasarela de pago. Si el pago se acepta, la reserva queda

confirmada y se publica el evento correspondiente. Si se rechaza, queda registrada como rechazada. Y si la pasarela está caída o el circuit breaker está abierto, el sistema no se cuelga esperando: corta de inmediato, deja la reserva en estado pendiente de pago y devuelve un error claro al cliente, que puede reintentar más tarde.

Esta es una de las tácticas de resiliencia de R7. Lo importante es que en ningún caso la reserva se pierde ni queda en un estado inconsistente: una caída de la pasarela degrada el flujo de pago sin afectar al resto del sistema ni a la reserva del cliente. La decisión está en el ADR-006 y el ADR-022.

Vista de Despliegue

Esta vista muestra cómo los componentes del sistema se asignan a la infraestructura de ejecución: qué corre dónde, en qué puertos, qué datos persisten entre reinicios y qué necesita salir a internet.

[Click aquí para ver imagen](#)

Catálogo de elementos

Todo el sistema corre sobre un único host con Docker Compose, y se levanta con un solo comando. Tomamos esta decisión por R9, que pide portabilidad: cualquiera que tenga Docker puede levantar el sistema completo sin instalar nada más. No buscamos alta disponibilidad con varios hosts porque es un proyecto académico, no un despliegue productivo, y agregar orquestación multi-nodo habría sumado complejidad sin un beneficio real para lo que se evalúa. La decisión está en el ADR-020.

El sistema está formado por trece contenedores. El **api-gateway** (nginx) es el único que expone su puerto al exterior, en el 8000 del host. Los tres servicios HTTP corren en sus puertos internos (3001, 3002 y 3003) pero no los exponen al host: solo se puede llegar a ellos a través del gateway, lo que evita que alguien saltee la autenticación entrando directo a un servicio. El ai-worker corre sin puerto, porque solo consume de colas. El frontend-geo, la aplicación de mapas complementaria, se expone en el 4173.

La infraestructura la forman postgres con PostGIS y pgvector, redis, rabbitmq, ollama y ors (el motor de rutas de OpenRouteService). Estos sí exponen sus puertos al host, pero solo para que el equipo pueda conectarse con herramientas de desarrollo durante la construcción, no porque el sistema lo necesite en runtime. La observabilidad la dan prometheus en el 9090 y grafana en el 3000.

Nodo	Características	Descripción
api-gateway (nginx)	Puerto host: 8000; imagen oficial nginx	Punto de entrada único
booking-service	Puerto interno: 3001; imagen propia	Reservas y clientes
operations-service	Puerto interno: 3002; imagen propia	Operaciones
tracking-service	Puerto interno: 3003; imagen propia	Traslados y GPS
ai-worker	Sin puerto; imagen propia	Clasificación asíncrona
frontend-geo	Puerto host: 4173; imagen propia	Mapa complementario
postgres	Puerto host: 5432; volumen persistente; imagen oficial	Base de datos (PostGIS + pgvector)
redis	Puerto host: 6379; volumen persistente; imagen oficial	Cache, colas y Pub/Sub

rabbitmq	Puertos host: 5672, 15672; volumen persistente; imagen oficial	Eventos entre servicios
ollama	Puerto host: 11434; volumen persistente (modelos ~GB); imagen oficial	IA local (embeddings y LLM)
ors	Puerto host: 8080; volumen persistente (grafos de ruta); imagen oficial	Geolocalización local
prometheus	Puerto host: 9090; imagen oficial	Recolección de métricas
grafana	Puerto host: 3000; volumen persistente (config); imagen oficial	Dashboards

Conector	Características	Descripción
Red Docker interna	TCP/IP; síncrono y asíncrono	Comunica todos los contenedores dentro del host
Puerto expuesto al host	TCP/IP; accesible desde exterior	Solo api-gateway (8000) y puertos de desarrollo de infra
HTTPS externo	TCP/IP; TLS; síncrono	Desde clientes externos hacia el api-gateway

Persistencia y conexión externa

Los datos que tienen que sobrevivir a un reinicio viven en volúmenes Docker nombrados: las tablas de Postgres, el journal de Redis, los mensajes pendientes de RabbitMQ, los modelos descargados de Ollama (que pesan varios gigabytes y no queremos volver a bajar cada vez), los grafos de ruta de ORS, y la configuración de Grafana. Tenerlos en volúmenes nos permite reiniciar el sistema sin perder datos.

Un punto que conviene aclarar, porque lo verificamos al auditar nuestro propio sistema: el único servicio externo que realmente sale a internet es Firebase, para validar los tokens de autenticación. La pasarela de pago es un adapter mock que corre dentro del sistema y no necesita conexión externa. ORS y Ollama también son locales, corren como contenedores propios sin depender de servicios en la nube. Esto hace que el sistema sea casi autocontenido, lo que ayuda a la portabilidad de R9.

Una última aclaración: el gps-simulator no es uno de los trece contenedores. Es una herramienta de línea de comandos que corremos por separado cuando queremos generar tráfico GPS para probar o para la demo, así que no forma parte del despliegue permanente del sistema.

Vista de Instalación

Esta vista muestra los artefactos que efectivamente se instalan para que el sistema corra. En nuestro caso, los artefactos de instalación son imágenes Docker.

[Click aquí para ver imagen](#)

Las imágenes de los seis componentes que escribimos nosotros (los tres servicios HTTP, el ai-worker, el gateway y el frontend) se construyen desde un Dockerfile propio de cada uno. El archivo docker-compose.yml es el que orquesta todo: define qué imagen corre en cada contenedor, en qué orden arrancan (con dependencias y health checks encadenados), qué puertos se exponen y qué volúmenes se montan. Las imágenes de la infraestructura

(postgres, redis, rabbitmq, ollama, ors, prometheus y grafana) no las construimos: se toman directamente de registros públicos, así que para esas el artefacto es la referencia a la imagen oficial en el docker-compose.

Decisiones de arquitectura (ADRs)

A lo largo del diseño tomamos veinticinco decisiones de arquitectura que registramos como ADRs. Aquí desarrollamos las nueve que más impacto tuvieron sobre los atributos de calidad, siguiendo el formato de contexto, decisión, justificación, estado y consecuencias. El resto está en el anexo como tabla resumen.

ADR-001: Arquitectura híbrida de tres servicios más workers

El requerimiento de disponibilidad (R7) exige que el grupo de funcionalidades de reservas y pagos (F2 a F6) y el grupo del traslado en curso (F13 a F15) sigan operando aunque otra parte del sistema falle. A esto se suma una restricción de proyecto: somos un equipo de tres personas con un tiempo acotado.

Decisión: Separamos el sistema en tres servicios HTTP, cada uno en su propio contenedor, más dos procesos worker y un API gateway como punto de entrada único. Compartimos una sola instancia de base de datos con un esquema separado por servicio.

Justificación: Descartamos el monolito porque una falla en el procesamiento de GPS, que es la parte más exigida, podía tumbar también el flujo de pagos, que es justo lo que R7 prohíbe. Descartamos los microservicios puros con base de datos por servicio porque introducían transacciones distribuidas y patrones Saga, cuya complejidad excedía lo que el equipo podía sostener en el plazo del proyecto. La arquitectura híbrida da el aislamiento que pide R7 sin el costo de la consistencia distribuida.

Estado: Aceptado.

Consecuencias: Ganamos aislamiento de fallas y la posibilidad de escalar cada servicio por separado. Como contrapartida, sacrificamos simplicidad: tenemos más contenedores que mantener, comunicación asíncrona entre servicios, y un contenedor de inyección de dependencias en cada uno. La base de datos compartida sigue siendo un punto único de falla a nivel de infraestructura.

ADR-002: Pipes and Filters para el pipeline GPS

R3 exige procesar cada punto GPS de punta a punta, incluida la detección de situaciones, en menos de cinco segundos. El procesamiento tiene etapas bien diferenciadas (validar, enriquecer, persistir, detectar zona, detectar parada, generar alerta, publicar) que conviene poder modificar de forma independiente.

Decisión: Modelamos el procesamiento de GPS como una secuencia de siete filtros independientes encadenados por colas, donde cada filtro hace una sola cosa y pasa el resultado al siguiente.

Justificación: La alternativa era un único bloque de código que hiciera todo el procesamiento. La descartamos porque ataba las manos para modificar o agregar una etapa sin tocar las demás, y porque mezclaba responsabilidades. Con Pipes and Filters podemos cambiar la lógica de una etapa, o agregar un nuevo tipo de detección, sin afectar al resto.

Estado: Aceptado.

Consecuencias: Ganamos modificabilidad y un procesamiento desacoplado de la recepción del punto. El costo es la complejidad de orquestar siete colas y de manejar los errores entre pasos.

ADR-003: Bull para colas internas y RabbitMQ para eventos entre servicios

El sistema necesita dos tipos de mensajería distintos: encadenar etapas dentro de un mismo servicio (el pipeline GPS, los trabajos de IA) y comunicar eventos entre servicios distintos.

Decisión: Usamos Bull sobre Redis para los pipelines internos de un servicio, y RabbitMQ para los eventos entre servicios.

Justificación: Bull no resuelve el fan-out de un evento hacia varios servicios consumidores, que es lo que necesitamos entre servicios. RabbitMQ, a su vez, no es eficiente para un pipeline local de alta frecuencia como el de GPS. Cada herramienta queda donde rinde mejor.

Estado: Aceptado.

Consecuencias: Ganamos una separación clara entre la mensajería interna y la externa. El costo es operar dos tecnologías de mensajería en lugar de una.

ADR-006: Circuit breaker sobre la pasarela de pago

R7 exige que el pago (F6) no bloquee al resto del sistema si la pasarela externa falla. Una pasarela lenta o caída no puede dejar al cliente esperando indefinidamente ni propagar la falla.

Decisión: Envolvemos la pasarela de pago en un circuit breaker que corta rápido cuando detecta fallas, y dejamos la reserva en estado pendiente de pago para que el cliente reintente más tarde.

Justificación: Descartamos reintentar automáticamente desde una cola, porque ante una caída prolongada eso genera una avalancha de reintentos que empeora el problema. El circuit breaker falla rápido y de forma controlada, que es lo que R7 necesita.

Estado: Aceptado.

Consecuencias: El flujo de pago no se cuelga ante una caída de la pasarela y la reserva nunca queda en un estado inconsistente. Como contrapartida, cuando el breaker está abierto el pago no está disponible y el cliente tiene que reintentar manualmente.

ADR-007: Clasificación con IA en cascada de estrategias

R10 nos pide evaluar y comparar tres estrategias de clasificación del bien (búsqueda semántica, IA generativa local y una tercera a criterio del equipo) por tiempo de respuesta, precisión y simplicidad. Además, R1 limita el tiempo de respuesta de la creación de reservas.

Decisión: Pusimos las tres estrategias detrás de una interfaz común y las usamos en cascada, de la más barata a la más cara: primero reglas por palabras clave, después búsqueda semántica con embeddings, y por último el modelo de lenguaje. Las dos primeras corren de forma síncrona dentro del request; el modelo de lenguaje corre asíncrono en el ai-worker, solo cuando las anteriores no alcanzaron.

Justificación: Usar siempre el modelo de lenguaje era más simple de programar pero demasiado lento para R1 (su latencia mediana medida fue de 5.5 segundos). Las reglas son instantáneas pero fallan con sinónimos, y los embeddings dan el mejor balance pero no resuelven todos los casos. La cascada usa cada estrategia donde rinde. El detalle de las mediciones está en el anexo.

Estado: Aceptado.

Consecuencias: La mayoría de las reservas se clasifican rápido y de forma síncrona, y solo las que ninguna estrategia rápida resuelve pagan el costo del modelo de lenguaje, sin bloquear al cliente. El costo fue desarrollar tres implementaciones más la orquestación del flujo asíncrono.

ADR-009: Una base de datos compartida con un esquema por servicio

Teníamos que decidir si cada servicio tendría su propia base de datos o si compartirían una. La decisión impacta el aislamiento de datos y la complejidad de las operaciones que cruzan dominios.

Decisión: Compartimos una sola instancia de PostgreSQL con un esquema separado por servicio y pools de conexión independientes. Cada servicio solo accede a su propio esquema.

Justificación: Una base por servicio daba más autonomía, pero a cambio de transacciones distribuidas para las operaciones que cruzan dominios, lo cual era desproporcionado para el alcance del proyecto. La base compartida con esquemas separados da el aislamiento lógico que necesitamos sin esa complejidad.

Estado: Aceptado.

Consecuencias: Ganamos simplicidad y velocidad de desarrollo. Como contrapartida, la base es un punto único de falla a nivel de infraestructura, y el almacenamiento queda acoplado, aunque mitigado porque cada servicio solo toca su esquema.

ADR-012 y ADR-017: Notificación al operador con SSE y fan-out por Redis

F19 exige notificar al operador en menos de un segundo cuando llega una reserva que el sistema no pudo clasificar. Además, R8 pide poder escalar el servicio horizontalmente, y las conexiones de notificación viven en memoria de la instancia que las atiende.

Decisión: Usamos Server-Sent Events para empujar las notificaciones al operador, y un canal de Redis Pub/Sub para hacer el fan-out entre las instancias del booking-service.

Justificación: Elegimos SSE porque es la forma más simple de empujar datos del servidor al cliente con Express nativo, sin la complejidad de WebSocket que no necesitábamos al ser comunicación en un solo sentido. Sumamos el fan-out por Redis porque, al escalar el servicio para R8, una instancia que genera el evento no podría notificar a un operador conectado a otra instancia; con Pub/Sub, la instancia que genera publica en un canal y todas las suscritas empujan a sus conexiones.

Estado: Aceptado.

Consecuencias: El operador recibe las notificaciones en tiempo real sin importar a qué instancia esté conectado, lo que permite escalar el booking-service. El costo es que Redis pasa a ser una dependencia crítica también para las notificaciones.

ADR-018: Bloqueo pesimista para la asignación de vehículos

F12 tiene una condición de carrera: dos operadores podrían asignar el mismo vehículo al mismo tiempo, lo que dejaría el sistema en un estado inválido que la letra prohíbe.

Decisión: Resolvemos la asignación con un bloqueo pesimista a nivel de base de datos, tomando la fila del vehículo con un SELECT FOR UPDATE dentro de una transacción.

Justificación: La alternativa optimista (detectar el conflicto recién al guardar) era más liviana, pero dejaba una ventana de tiempo donde dos asignaciones podían colarse. Como la letra exige impedir la doble asignación, preferimos el bloqueo pesimista que la previene de raíz: si dos operadores asignan a la vez, el segundo queda bloqueado hasta que el primero confirma, y entonces ve que el vehículo ya no está disponible.

Estado: Aceptado.

Consecuencias: Garantizamos que un vehículo no se asigne dos veces. El costo es una pequeña espera del segundo operador en el caso poco frecuente de asignaciones simultáneas sobre el mismo vehículo.

ADR-022: Ports y Adapters para los servicios externos

R5 exige poder simular los servicios externos en las pruebas, y F6 establece que el proveedor de pagos puede cambiar. El sistema integra autenticación, pagos, geolocalización, IA y correo.

Decisión: Todos los servicios externos se usan a través de una interfaz (Port) que define el dominio, con adapters intercambiables en la capa de infraestructura. Cambiar de proveedor es agregar un adapter nuevo y cambiar una línea de configuración, sin tocar la lógica de negocio.

Justificación: Esta decisión responde directamente a dos requerimientos. Para R5, en las pruebas inyectamos adapters mock en lugar de los reales. Para F6, el Port IPaymentGateway permite cambiar de proveedor de pago sin tocar el dominio. Consultamos a la cátedra y confirmó que los proveedores externos pueden ser implementados por el equipo o tomados de un proveedor existente, mientras sean externos al sistema, por lo que el adapter mock de pago es una decisión de alcance válida para esta entrega.

Estado: Aceptado.

Consecuencias: Ganamos modificabilidad (cambio de proveedor sin tocar negocio) y testabilidad (mocks inyectables). El costo es mantener una interfaz más un adapter real más un mock por cada servicio externo.

Sobre el proveedor de pagos

La letra pide una pasarela externa y aclara que el proveedor puede cambiar. Nuestra respuesta arquitectónica fue construir la infraestructura de Ports y Adapters que permite acoplar distintos proveedores, de manera que si uno falla podemos enchufar otro sin tocar el dominio. Para este obligatorio implementamos un adapter mock de la pasarela, que simula los casos de pago aceptado, rechazado y de pasarela caída, porque eso nos permitía demostrar todo el comportamiento del flujo y la táctica del circuit breaker sin depender de una cuenta de un proveedor real. Consultamos el foro y vimos que los proveedores externos pueden ser implementados por el equipo o tomados de un proveedor existente, mientras sean externos al sistema, así que nuestro enfoque es una decisión de alcance válida. Conectar un proveedor real es agregar un adapter más detrás de la misma interfaz.

Cómo el sistema resiste las fallas

R7 fue el requerimiento que más moldeó la arquitectura, así que vale la pena juntar en un solo lugar cómo se sostiene la disponibilidad de los flujos críticos ante distintas fallas.

La táctica de fondo es el aislamiento. Cada servicio corre en su propio contenedor con su propio pool de conexiones, de modo que un crash o una saturación en uno no se propaga a los demás. Sobre eso, la comunicación entre servicios es siempre asíncrona por RabbitMQ, lo que significa que los flujos síncronos (crear una reserva de empresa, cotizar, pagar, consultar) no dependen de que RabbitMQ ni de que otro servicio estén vivos. Si RabbitMQ se cae, esos flujos siguen funcionando; lo único que se degrada son los flujos asíncronos, como la notificación al operador o la actualización de las proyecciones, y aún esos no pierden datos porque las reservas quedan persistidas con su estado y las alertas se guardan antes de intentar publicarlas.

Para los servicios externos usamos circuit breakers, que cortan rápido cuando un proveedor falla en lugar de dejar al sistema colgado esperando un timeout. El caso del pago es el más claro: si la pasarela cae, la reserva queda recuperable en estado pendiente y el cliente recibe un error accionable. Para la geolocalización agregamos degradación: si OpenRouteService no responde, el cálculo de distancia cae a una fórmula geométrica simple, de modo que la cotización sigue funcionando con una precisión un poco menor en lugar de fallar.

La siguiente tabla resume los tradeoffs de las decisiones principales, porque toda decisión de arquitectura favorece un atributo de calidad a costa de otro.

<u>Decisión</u>	<u>Atributo favorecido</u>	<u>Atributo sacrificado</u>
Tres servicios aislados (ADR-001)	Disponibilidad, Escalabilidad	Simplicidad
Pipes and Filters en GPS (ADR-002)	Performance, Modificabilidad	Simplicidad
Bull más RabbitMQ (ADR-003)	Separación de responsabilidades	Complejidad operativa
Circuit breaker en pagos (ADR-006)	Disponibilidad	Funcionalidad degradada ante falla
Cascada de IA (ADR-007)	Performance, Precisión	Complejidad de desarrollo
Base compartida con esquemas (ADR-009)	Simplicidad	Autonomía, punto único de falla
SSE con fan-out Redis (ADR-012, ADR-017)	Performance, Escalabilidad	Dependencia crítica de Redis
Docker Compose (ADR-020)	Portabilidad	Sin auto-recuperación nativa
Ports y Adapters (ADR-022)	Modificabilidad, Testabilidad	Más código por servicio externo

Video de demostración (demo)

El video de demo muestra el flujo completo de la plataforma MOVE ejecutado sobre el entorno local con Docker Compose levantado (13 contenedores). Todas las interacciones se realizan desde la colección Postman (docs/postman/move-collection.json) con el environment MOVE Platform local y datos precargados por el script `pnpm seed:demo`. El video está estructurado en cinco bloques según los roles del sistema: introducción de la infraestructura activa, gestión de categorías, zonas, vehículos y usuarios por parte del administrador, registro e ingreso de clientes, creación de reservas con cascade de clasificación IA, cotización automática, pago con circuit breaker y consulta de reservas con filtros, clasificación manual de reservas por el operador con envío de email de rechazo al cliente, asignación de vehículo y conductor con validación de doble booking, recepción de puntos GPS con pipeline P1–P7, detección de situaciones, generación de alertas y finalización de traslado, cerrando con métricas en Grafana, endpoint `/metrics` y logs estructurados en JSON.

Link al video: <https://www.youtube.com/watch?v=dCr6xqgtfuQ>

Anexo

Catálogo completo de ADRs

El cuerpo desarrolla las nueve decisiones de mayor impacto. La tabla lista las veinticinco, con su decisión y el atributo de calidad que la motivó.

ADR	Decisión	Atributo
ADR-001	Arquitectura híbrida de tres servicios más dos workers	Disponibilidad, Escalabilidad
ADR-002	Pipes and Filters con Bull para el pipeline de tracking	Modificabilidad
ADR-003	Bull para colas internas y RabbitMQ para eventos entre servicios	Separación de responsabilidades
ADR-004	Redis para estado caliente del vehículo y cache del top-20	Performance
ADR-005	PostgreSQL con PostGIS para GPS	Modificabilidad
ADR-006	Strategy más circuit breaker para la pasarela de pago	Disponibilidad, Modificabilidad
ADR-007	Strategy para la clasificación con tres estrategias de IA en cascada	Modificabilidad, R10
ADR-008	Reglas de cotización en base de datos, modificables sin cambiar código	Modificabilidad
ADR-009	Base de datos compartida con un esquema por servicio	Simplicidad
ADR-010	Prisma para el grueso del acceso y pg crudo para las consultas PostGIS	Performance
ADR-011	Firebase Auth como servicio externo de autenticación	Seguridad, Simplicidad
ADR-012	SSE para la notificación en tiempo real al operador	Performance (F19 menos de 1s)
ADR-013	Prometheus más Grafana para observabilidad	Observabilidad (R4)
ADR-014	Servicio de correo con adapter de Nodemailer para el email de rechazo	Modificabilidad
ADR-015	F21 interpretada como un error de la letra (se lee como F19)	Clarificación
ADR-016	API gateway con nginx como punto de entrada único	Seguridad
ADR-017	Fan-out de SSE con Redis Pub/Sub para escalar booking horizontalmente	Disponibilidad, Escalabilidad
ADR-018	Bloqueo pesimista con SELECT FOR UPDATE en la asignación de vehículo	Integridad, Disponibilidad
ADR-019	Colas Bull por dispositivo y hashing consistente para preservar el orden GPS	Integridad, Escalabilidad
ADR-020	Docker Compose como único mecanismo de despliegue	Deployabilidad (R9)
ADR-021	Estrategia de testabilidad con simulador GPS y mocks inyectables	Testabilidad (R5)

ADR-022	Ports y Adapters para todos los servicios externos	Modificabilidad
ADR-023	Proyecciones de eventos para los datos que cruzan servicios	Integridad , Seguridad
ADR-024	Cache de zonas en Redis con turf.js para el geofence	Performance, Seguridad
ADR-025	Cache de vehículos en tracking alimentado por evento de registro	Performance, Seguridad

Comparativa de las tres estrategias de clasificación (R10)

R10 pide evaluar y comparar tres estrategias para clasificar el bien a partir de su descripción, midiendo tiempo de respuesta, precisión y simplicidad. Las tres están implementadas y funcionando. Medimos sobre un conjunto de prueba de veinte descripciones reales.

Criterio	Reglas por palabras clave	Embeddings (búsqueda semántica)	Modelo de lenguaje (qwen2.5:3b)
Latencia p50	0.13 ms	37 ms	5.5 s en CPU
Precisión	25%	65%	45%
Dependencias en runtime	Ninguna (todo en memoria)	Ollama para el embedding	Ollama para el modelo
Simplicidad	Alta	Media	Media, pero salida variable

El resultado más interesante, fue que la búsqueda semántica con embeddings (65% de precisión) superó al modelo de lenguaje usado solo (45%) en nuestro conjunto de prueba. Aprendimos también que la precisión de los embeddings dependía sobre todo de la calidad del texto con el que generábamos los vectores de cada categoría: al principio usábamos solo palabras clave cortas y la precisión era del 5%, y al pasar a usar las descripciones ricas de las categorías saltó al 65% sin cambiar el algoritmo.

Por eso ninguna estrategia sola alcanzaba. Las reglas son instantáneas pero fallan con sinónimos. Los embeddings dan el mejor balance pero un tercio de los casos no supera el umbral. El modelo de lenguaje solo es demasiado lento (5.5 segundos de mediana, y hasta 300 segundos en un caso bajo carga en CPU) y violaría el tiempo de respuesta que pide R1. La cascada usa cada una donde rinde: las reglas y los embeddings resuelven la mayoría rápido y de forma síncrona, y el modelo de lenguaje queda como último recurso asíncrono para los casos límite, que es exactamente por lo que corre fuera del request del cliente. Esta variabilidad de latencia del modelo en CPU es la razón concreta por la que lo hicimos asíncrono.

Pruebas de carga con K6

Para no quedarnos en la intuición de que el sistema cumple los tiempos que pide la letra, usamos K6, una herramienta de pruebas de carga que simula muchos usuarios golpeando el sistema en simultáneo y mide cuánto tarda cada respuesta. La elegimos porque es la que sugiere la propia letra y porque permite definir escenarios que reproducen las condiciones de carga que describe cada requerimiento no funcional de performance.

Armamos un escenario de K6 por cada requerimiento que queríamos verificar. Uno para la creación de reservas de empresa (R1), otro para las consultas y listados bajo carga (R2),

otro para la recepción de puntos GPS (R3), y otro para el escalado a alta demanda (R8), que aplica varias veces la carga base para ver si el sistema la absorbe. Cada escenario hace login con usuarios de prueba y después hace las peticiones que corresponden, registrando los percentiles de latencia y la tasa de error. Los resultados de estas mediciones, con los números concretos, están en la sección de evidencia de performance del cuerpo del documento.

Un punto importante es que K6 no puede correr contra un sistema vacío. Para que las peticiones tengan sentido (crear una reserva necesita un cliente registrado, consultar traslados necesita traslados existentes, enviar un punto GPS necesita un conductor y un dispositivo válidos), la base de datos tiene que estar cargada con datos de prueba antes de correr los escenarios. Por eso el flujo siempre es el mismo: primero se levanta el sistema, después se carga la base con un script de seed, y recién entonces se corren los escenarios de K6.

Carga de datos: los scripts de seed

Para cargar la base de datos usamos scripts de seed, que son programas que crean datos de prueba de forma automática y repetible, en vez de tener que cargarlos a mano cada vez. Tenemos dos scripts de seed con propósitos distintos.

Uno es el seed de demostración, pensado para tener el sistema con un conjunto de datos coherente y realista a la hora de mostrarlo: clientes de los dos tipos, conductores, vehículos, zonas geográficas, categorías y reservas en distintos estados. Lo usamos para la demo y para probar los flujos a mano. Aclaramos que todavía estamos terminando de definir cuál va a ser el conjunto exacto de datos de la demo final, así que lo describimos por lo que hace y no por su contenido puntual.

El otro es el seed de K6, pensado específicamente para las pruebas de carga. Crea los usuarios y datos que los escenarios de K6 necesitan para loguearse y operar, y deja registradas las credenciales de esos usuarios en un único archivo que los escenarios leen al correr. De esa forma, si cambia algún usuario de prueba, se cambia en un solo lugar y todos los escenarios siguen funcionando. Este seed es el que prepara el terreno para que K6 pueda hacer sus peticiones, como explicamos en el anexo anterior.

Ambos scripts se pueden volver a correr sin romper lo que ya existe, y tienen una opción de limpieza para empezar desde cero cuando hace falta. La idea de fondo es que cualquiera del equipo, o el corrector, pueda dejar la base en un estado conocido con un solo comando, sin depender de datos cargados a mano que nadie sabe de dónde salieron.

Gestión del proyecto y control de versiones

Antes de escribir una línea de código armamos un plan general del proyecto. Ahí pensamos las tecnologías que íbamos a usar y por qué, hicimos un boceto de las decisiones de arquitectura que terminaron siendo los ADRs, y definimos cómo se descomponía el sistema en servicios. Ese plan fue la base de todo lo demás: nos dio una visión común del rumbo antes de empezar a construir, en lugar de ir tomando decisiones sueltas sobre la marcha.

A partir de ese plan armamos un segundo nivel de planificación más concreto: lo dividimos en una gran cantidad de issues, más de ochenta a lo largo del proyecto, cada una con una tarea acotada. Repartimos esas issues entre los tres integrantes buscando que pudiéramos trabajar en paralelo: en la medida de lo posible, cada uno tomaba issues que no dependían de las de los demás, de modo que el avance de uno no bloqueara ni rompiera el trabajo del otro. Esto encajó naturalmente con la descomposición en servicios, porque cada integrante pudo hacerse cargo de un servicio o de un conjunto de funcionalidades sin pisarse con los demás.

Para llevar ese trabajo al repositorio seguimos un flujo de ramas inspirado en GitFlow. La idea central es que nadie trabaja directamente sobre la rama principal: cada issue vive primero en su propia rama y recién se integra cuando está lista y revisada. Tenemos una rama de integración, develop, donde se juntan todos los cambios a medida que se completan, y la rama main queda reservada para el estado estable. Cada funcionalidad, corrección o tarea de mantenimiento se hace en una rama propia que sale de develop, con un nombre que dice de qué se trata, por ejemplo una rama de feature para una funcionalidad nueva o una de fix para corregir un problema. Cuando el trabajo de esa rama está terminado, se abre un Pull Request hacia develop, que tiene que pasar la integración continua (que corre el lint, el chequeo de tipos y los tests) y ser revisado antes de integrarse.

Este flujo nos dio dos cosas. Por un lado, develop siempre quedó en un estado integrable, porque nada entra sin pasar por el Pull Request y sus controles. Por otro, el historial quedó ordenado y trazable: cada rama y cada Pull Request cuentan qué se hizo y por qué, lo que es especialmente útil en un equipo de tres personas trabajando en paralelo sobre servicios distintos. Los commits siguen una convención de tipo y descripción breve en español, de modo que el historial se lee como un relato de lo que fue cambiando en el proyecto.

Uso de agentes de inteligencia artificial

Durante el proyecto usamos asistentes de inteligencia artificial generativa, principalmente Claude, como una herramienta más de trabajo. Conviene ser claros sobre cómo los usamos, porque la diferencia entre que la herramienta trabaje para nosotros y que trabaje en nuestro lugar está justamente en quién toma las decisiones, y en este proyecto las decisiones de arquitectura las tomamos nosotros.

Los usamos en cuatro frentes. En la planificación, para ordenar el trabajo en fases, descomponer las tareas y discutir enfoques antes de empezar a programar. Ese trabajo se materializó en más de ochenta issues en GitHub, gestionadas a través de un GitHub Project: la IA nos ayudó a descomponer las fases en tareas concretas, que luego cargamos como issues y distribuimos entre los integrantes para poder trabajar en paralelo. En el código, para acelerar la escritura una vez que ya habíamos decidido qué construir y cómo: el agente generaba código siguiendo las decisiones que habíamos tomado, y nosotros lo revisábamos, lo ajustábamos y lo integrábamos. En la investigación, para comparar alternativas técnicas y entender los compromisos de cada opción antes de elegir, por ejemplo al evaluar las tres estrategias de clasificación de R10 o al decidir entre bloqueo optimista y pesimista para la asignación de vehículos. Y en la revisión, para auditar el código en busca de inconsistencias y verificar que lo que decía nuestra documentación coincidiera con lo que el código realmente hacía. Esa última tarea fue particularmente útil: nos ayudó a encontrar varios lugares donde nuestra propia documentación interna se había adelantado al código, y los corregimos antes de escribir este documento.

Deuda técnica y mejoras identificadas

Al auditar nuestro propio sistema antes de documentarlo, encontramos algunas cosas que quedaron por mejorar, y preferimos nombrarlas acá en lugar de esconderlas.

La primera es la configuración de los circuit breakers de autenticación. Tal como están, abren con un solo error transitorio, porque no fijamos un volumen mínimo de peticiones antes de evaluar el porcentaje de fallas. En la práctica, un error puntual de red contra Firebase puede abrir el breaker y bloquear los registros durante treinta segundos. La mejora es configurar ese volumen mínimo.

La segunda es que de los consumidores de eventos de RabbitMQ, solo dos tienen configurada una cola de mensajes muertos para los que fallan al procesarse; el resto descarta el mensaje ante un error.

La tercera es menor: una de las etapas del pipeline GPS recibe el mismo punto dos veces por un doble encolado, lo que puede hacer que la detección de parada se procese de forma duplicada. El resultado final no se ve afectado, porque aunque el punto se procese dos veces, el sistema no crea la misma alerta dos veces.

Ninguna de estas tres compromete los flujos críticos ni los resultados del sistema, pero las dejamos anotadas como lo primero a atacar por si alguien continuará el proyecto.